DaGOAL

CAPSTONE DEFENSE REVIEWER

A Gamified Daily Activity Planner for Screen Time Reduction

Application walkthrough • Research findings • Panel practice • Honest limitations

Proponents
Altheah Y. Gabas • Marlou Jewel A. Naje • Shaun Lee F. Ocana • Jheru G. Rosario

BS Information Technology
STI College Pasay-EDSA • November 2026

Purpose of this guide
A practical prep document combining the approved capstone print copy with a review of the current DaGOAL Android project code. It covers what the application implements, what the study reports, and how to answer likely panel questions without overstating the evidence.

Defense date: Saturday, November 7, 2026

Use your own words and agree on manuscript-specific answers as a team.

How to use this reviewer

Practice the short explanations first, then use the Q&A; as a mock-panel script. When you do not know a detail, state what you know, mark the boundary, and offer to verify it from the project documentation.

- **Application facts** below come from the Android project source inspected for this guide.
- **Suggested responses** are speaking templates, not claims of measured effectiveness.
- Replace every bracketed item with the exact wording/data from your manuscript.

One-sentence answer pattern: answer directly, name the mechanism, then mention the relevant condition or limitation.

Quick identity statement

DaGOAL is an Android goal-building application that turns selected real-life activities into daily quests, tracks progress through manual input or supported device signals, and uses XP, gold, levels, streaks, achievements, and avatar/shop rewards to encourage continued engagement. This describes implemented features; use your manuscript to state the intended population and study objective.

At-a-glance

Area	What the app implements
Platform	Native Android app, Java source; min SDK 29, target SDK 36, version 1.0.
Main experience	Onboarding/profile, avatar customization, daily quest reveal, dashboard with Quest, Wardrobe, Shop, and Me sections.
Progression	Quest rewards feed local XP and gold; XP can advance levels. The app also tracks streaks, achievements, inventory, and clothing.
Persistence	SQLite is the primary structured store; SharedPreferences stores onboarding/settings state. Firebase Authentication and selected Firestore features support accounts/cloud functions.
Device integration	Step sensor foreground service; usage access plus overlay permission for app-avoidance quests; scheduled notification receiver.

1. Product purpose and feature story

The approved project frames DaGoal as a gamified daily activity planner intended to help users reduce excessive screen time by replacing passive use with physical, mental, and creative activities. The code implements generated and custom quests, progression/rewards, optional step tracking, and user-started screen-avoidance quests.

Defense-ready purpose statement

“The main purpose of DaGoal is to encourage users to reduce excessive screen time by replacing passive digital consumption with productive real-world activities. It generates daily quests informed by user preferences and supports follow-through with flexible task choices, progress tracking, XP, gold, streaks, achievements, and avatar rewards. Our initial study evaluated functionality and user perceptions after a three-day trial; it does not establish long-term screen-time reduction.”

Explain the design in three layers

- **Goal layer:** personalized daily quests across physical, mental, and creative activities, alongside custom/recurring quests.
- **Tracking layer:** manual confirmation, repetition/count progress, timed avoidance, and sensor-supported step tracking.
- **Motivation layer:** XP/gold, levels, streaks, achievements, consumables, cosmetic shop items, and avatar customization.

The manuscript describes selected-app blocking. The current code starts an avoidance timer and monitors foreground-app usage, then shows an interruption overlay with options. Describe that observed implementation accurately; do not promise an OS-level hard block.

2. User journey

Stage	What to say
Launch	LoadingActivity prepares the local database and preloads some app/avatar assets. It routes returning users to dashboard or daily reveal, and new users to the welcome flow.
Entry	MainActivity provides guest avatar creation and a route to login/authentication options.
Set up	AvatarCreationActivity customizes the avatar. ProfilingActivity collects onboarding questionnaire answers/profile data. AppSelectionActivity lets users choose apps for avoidance monitoring.
Daily start	DailyRevealActivity is used when the app detects that a new day’s quests need to be generated/revealed.
Use	DashboardActivity hosts Quest, Wardrobe, Shop, and Me sections. Users complete quests, manage items/avatar, and inspect progress.
Support	SettingsActivity covers account, notification, privacy/permissions, sound, and about settings; StreakActivity and StreakInfoActivity display streak details.

Short user-flow explanation

“A new user enters through the welcome/authentication flow or starts as a guest, customizes an avatar, answers onboarding questions, and can select apps for avoidance quests. On later launches, the app prepares the day’s quests and opens the dashboard. Completing quests updates progress and rewards, while the other dashboard sections support avatar items, shop purchases, achievements, and profile/streak review.”

3. Activities and components: what each one does

Use "Activity" to mean an Android screen/controller. Some product areas are dashboard views rather than separate Activities; say that distinction accurately.

Activity	Defense-ready purpose
LoadingActivity	Startup coordinator: initializes/opens the local database, preloads assets, checks onboarding/login/guest state, and routes to the next screen.
MainActivity	Welcome/entry screen. Offers guest avatar creation and login navigation.
AuthOptionsActivity	Presents available authentication choices.
AuthEmailActivity	Email-based authentication flow, including sign-in/registration paths.
AuthPasswordActivity	Password entry/authentication step and Firebase Auth sign-in handling.
SignUpActivity	Creates an account through Firebase Authentication and initializes selected cloud profile data.
AvatarCreationActivity	Lets users customize avatar features/colors, renders a preview/snapshot, saves avatar configuration locally, and merges selected avatar fields to Firestore for an account.
ProfilingActivity	Runs the onboarding questionnaire, records answers/preferences and profile information locally, and syncs selected profile state when signed in.
AppSelectionActivity	Allows selection of installed apps for avoidance-related quests; explain usage-access and package-visibility requirements carefully.
DailyRevealActivity	Daily quest introduction/reveal screen, entered when a new daily quest set is due.
DashboardActivity	Primary workspace hosting Quest, Wardrobe, Shop, and Me areas, quest interactions, progression summaries, and navigation.
StreakActivity	Shows a daily streak popup/summary and streak-related prompt.
StreakInfoActivity	Displays streak history/calendar information and related prediction/chest details.
SettingsActivity	Account, notification, privacy/permissions, sound/haptics, and about controls; includes account management actions.

Services and helpers: StepTrackingService listens to Android step sensors and updates step quest progress; AppMonitorService polls usage stats while avoidance quests are active; AvoidanceOverlayManager displays interruption/ignore UI; TaskManager contains quest generation, progress, completion, reward, streak, and related logic; DatabaseHelper creates/upgrades/seeds SQLite; DailyNotificationReceiver handles daily/reminder/boot events.

4. Core logic and data flow

Daily quest generation

TaskManager checks the date and streak state, preserves due recurring custom quests, determines quest slots from level (three base slots, one added per five levels, capped at ten), may include an avoidance quest when blocked apps exist, then fills remaining slots from survey-driven or other templates. Generated quests can vary by user state and random choices; do not promise an identical fixed list.

Quest completion and progression

When a quest is completed, TaskManager reads its stored reward, marks it complete, applies an active XP/gold boost if enabled, updates local gold and XP, calculates level-ups using configured thresholds, grants level-up rewards where applicable, and updates achievement/chest counters. The dashboard refreshes the visible state. Avoid citing a specific formula unless checked in the current implementation or demo build.

Step quest flow

User grants activity-recognition permission → StepTrackingService runs as a foreground service with a persistent notification → sensor readings are converted to a daily step count → TaskManager updates a matching step quest → progress appears on the dashboard. This depends on compatible device sensors and permissions.

Avoidance quest flow

User selects apps. If a daily avoidance quest is generated, DashboardActivity asks for Usage Access and overlay permission. The user starts the quest; AppMonitorService then checks recent foreground app usage and may show an interruption overlay. The user can leave, choose a timed ignore with reduced reward, or stop/reset the quest. This is opt-in Android monitoring, not a universal system-level blocker.

Storage and cloud boundary

Data/service	Role in this build
SQLite (DatabaseHelper)	Primary structured local store: user progression, daily quests, templates, inventory, consumables, achievements, blocked apps, preferences, and streak history.
SharedPreferences	Local flags/settings such as onboarding state, daily generation date, and selected preferences.
Firebase Authentication	Email account registration/sign-in flows. Guest use is also supported by local state.
Cloud Firestore	Selected account/profile/avatar/streak sync and online shop/chest functions are present. Do not claim the entire local quest database is cloud-backed without confirming a sync implementation.

5. Likely panel questions and suggested answers

Q. What is the main purpose of DaGOAL?

Suggested response: DaGoal is a gamified daily activity planner intended to help users reduce excessive screen time by replacing passive digital consumption with physical, mental, and creative activities. It uses preference-informed quests, flexible choices, progress feedback, and rewards to encourage participation.

Q. What problem are you trying to address?

Suggested response: The capstone focuses on excessive screen use and passive routines, particularly among students and young adults. DaGoal offers structured real-world activities as alternatives and optional screen-avoidance quests. Our evaluation measures initial user perceptions after a short trial, not a causal or long-term reduction in screen time.

Q. Who are the target users?

Suggested response: The manuscript identifies students and young adults as the intended audience. In the evaluation, respondents were reported as ages 19-26. The Android implementation supports Android API 29 and later; that device requirement is separate from the research population.

Q. How does DaGOAL differ from a basic to-do list?

Suggested response: It combines task tracking with XP, gold, levels, streaks, achievements, avatar customization, inventory, and a shop. It also has optional device-assisted step tracking and app-avoidance quest monitoring. Comparative effectiveness must be supported by study data.

Q. How are daily quests generated?

Suggested response: TaskManager creates a date-specific set using templates and user preference/profile signals, preserves due recurring custom quests, considers level-based quest slots, and may add an avoidance quest when apps were selected. Some choices are randomized.

Q. How are quests tracked?

Suggested response: Generic quests use manual confirmation; count/repetition goals update stored progress; supported walking goals use Android step sensors when permission and hardware are available; timed screen-avoidance quests are started by the user and rely on Usage Access plus overlay permission for interruption UI.

Q. How does a user earn XP and gold?

Suggested response: Each quest stores reward values. On completion, TaskManager applies eligible active boosts, adds XP and gold to the local user record, evaluates level thresholds, and updates achievements/reward counters.

Q. Why did you use gamification?

Suggested response: The design uses feedback and progression mechanics to make completion visible and give users in-app incentives. That is our design rationale. Whether it increases adherence or motivation is a research question answered only by our evaluation results.

Q. Where is data stored?

Suggested response: Most structured app state is local in SQLite, with UI/onboarding preferences in SharedPreferences. Firebase Authentication handles account identity, while Firestore is used for selected profile/avatar/streak and online features. It is a hybrid design; not every record is cloud-synchronized.

Q. Can it work offline?

Suggested response: Many core quest and progression operations are local, so they can work without network once set up. Authentication, cloud synchronization, and online shop functions depend on connectivity. Demonstrate exact offline behavior on the defense build.

Q. How do you protect privacy for app monitoring?

Suggested response: Monitoring is tied to an avoidance quest that the user starts and requires Android Usage Access; interruption overlays require overlay permission. The app uses a foreground service notification while monitoring. State the exact policy and data handling commitments from your approved privacy documentation.

Q. The manuscript says blocking. Does the current build hard-block selected apps?

Suggested response: The current code monitors recent foreground app usage during a user-started avoidance quest and presents an overlay with a go-home action or timed ignore/reset options. Describe this observed implementation accurately; do not promise guaranteed OS-level blocking.

Q. What if permission is denied or no step sensor exists?

Suggested response: The permission-dependent feature cannot fully operate. Step tracking needs a compatible sensor and activity-recognition permission. Avoidance monitoring needs Usage Access, and overlay presentation needs overlay permission. Other quest functions remain available; demonstrate the denied state on your build.

Q. How is streak calculated?

Suggested response: The app stores current streak, last completion date, longest streak, and streak history. TaskManager checks date transitions around daily quest generation/completion and updates streak state. Explain the qualifying rule from verified current behavior and demonstrate a scenario.

Q. What are the main screens?

Suggested response: DashboardActivity is the main workspace and hosts Quest, Wardrobe, Shop, and Me. Onboarding/authentication, avatar creation, profiling, app selection, daily reveal, streak details, and settings use separate Activities.

Q. What limitations did you identify?

Suggested response: Some features depend on Android permissions and sensors, selected cloud features require connectivity, and generated quests may vary. The current database upgrade path drops and recreates daily_tasks and task_templates, so preserving those records through migration is an important future improvement.

Q. How did you functionally test the application?

Suggested response: The manuscript reports 97 manual functional test cases covering startup/authentication, offline use, onboarding, quest generation and replacement, task verification, rewards, shop/wardrobe, profile/streaks, permissions, synchronization, step tracking, and avoidance behavior. All 97 were marked Passed under the documented test conditions. This is functional testing, not proof of long-term behavior change.

Q. How did you evaluate user perceptions?

Suggested response: Respondents used the app for three days, then completed an online software-quality questionnaire using a four-point agreement scale across usability, enjoyment/engagement, screen-time management, performance, and overall experience. The manuscript reports n=57 in the abstract/table, but its implementation-results text says 56 participants; reconcile the correct number before defense.

Q. What were the evaluation results?

Suggested response: The reported grand mean was 3.31 (SD=0.30), interpreted as Strongly Agree. Domain means: Usability 3.42 (SD .39, Strongly Agree); Enjoyment/Engagement 3.22 (.46, Agree); Screen Time Management 3.40 (.38, Strongly Agree); Performance 3.25 (.40, Agree); Overall Experience 3.26 (.41, Strongly Agree). These are respondent ratings, not objective usage-time measurements.

Q. Can you conclude that DaGoal reduced screen time?

Suggested response: Not definitively from this evaluation. The study lasted three days and relied mainly on participant feedback. The screen-time domain mean (3.40) indicates respondents felt the app raised awareness or encouraged better habits; it is not a measured before-and-after reduction or evidence of sustained effect.

Q. What sample-count issue should the team clarify?

Suggested response: The abstract and results table label the evaluation n=57, while the implementation-results paragraph says findings were based on 56 participants. Check the raw response export and final analyzed dataset, then correct the manuscript or explain exclusions and state the valid n consistently.

Q. What is the most important contribution?

Suggested response: The contribution is [exact manuscript wording]. In the software, it is embodied in [specific capability]. We avoid claiming novelty beyond what our related-work review supports.

Q. What would you improve next?

Suggested response: Priorities include safer non-destructive SQLite migrations, clearer permission fallback handling, broader device testing, and evaluation with intended users. Improvements should follow measured findings and participant feedback.

Q. What if you do not know an answer?

Suggested response: “I want to be precise. I can confirm [known fact]. The detail is not in the evidence I have at hand, so I would verify it against [code/manuscript/test record] rather than guess.”

6. Study design, reported results, and what they mean

Evaluation procedure

The manuscript reports distributing the APK to selected respondents, who used it for three days. They then completed an online software-quality questionnaire using a four-point scale (1 = Strongly Disagree to 4 = Strongly Agree). The evaluated domains were Usability, Enjoyment and Engagement, Screen Time Management, Performance, and Overall Experience.

Functional testing

The project reports 97 manual functional test cases, all marked Passed under the conditions used. Coverage included startup/account flows, offline access, onboarding, quest generation/replacement, manual/count/timed/step verification, rewards, shop and wardrobe, settings, permissions, sync, and background tracking. Present this as a test result, not proof that every device/environment is defect-free.

User evaluation results from the capstone print copy

Domain	Mean	SD	Interpretation
Usability	3.42	0.39	Strongly Agree
Enjoyment and Engagement	3.22	0.46	Agree
Screen Time Management	3.40	0.38	Strongly Agree
Performance	3.25	0.40	Agree
Overall Experience	3.26	0.41	Strongly Agree
Grand Mean	3.31	0.30	Strongly Agree

Interpretation to use at defense

“The results indicate positive initial acceptance and perceived usability after the three-day trial. Respondents gave Screen Time Management a 3.40 mean, suggesting they felt the app increased awareness or encouraged better habits. Because the study period was short and the measure was self-reported, we cannot conclude that screen time objectively decreased or that the effect persisted.”

Important manuscript consistency check

The abstract and table identify n=57, while the implementation-results paragraph states 56 participants. Confirm the correct analyzed sample using the raw response export and explain any exclusions. Use one consistent n in the defense slides and oral answer.

7. Technical and methodological limits to explain clearly

These are points to prepare for, not reasons to undermine the project. A measured, specific answer usually builds more confidence than an absolute claim.

Topic	Accurate framing
Android permission dependence	Activity recognition, Usage Access, overlay, and notification behavior varies by OS version/device settings. State the permission and feature dependency.
Sensor availability	Step tracking requires a compatible step counter/detector. Explain what your test devices supported.
Hybrid persistence	The app primarily uses local SQLite; selected cloud features use Firebase. Do not imply complete backup/sync unless your build and tests demonstrate it.
Database migration	DatabaseHelper.onUpgrade drops and recreates daily_tasks and task_templates. Existing quest/template records can be lost during upgrade; a preserving migration should be prioritized.
Monitoring semantics	Avoidance monitoring checks foreground app usage through Android UsageStats and may interrupt with an overlay. It is not a guaranteed hard block and can be affected by permissions/background restrictions/device behavior.
Efficacy vs implementation	A working feature demonstrates implementation. It does not prove better habits, health, productivity, or sustained behavior.
Research specifics	Population, sampling, instrument validity, response rate, evaluation method, and numerical findings must come from approved research documents.

Polished limitation answer

“One technical limitation is that some features depend on Android-level permissions and supported sensors, so behavior can vary by device. Our core quest state is mainly stored locally, while only selected account/profile and online functions use Firebase. We also identified that the current database upgrade path recreates daily quests and templates, so preserving these records through migration is a priority for future work.”

8. Live demo talk track

1. Start and identify

“This is the DaGOAL Android application. It turns user goals into trackable quests and shows progression through rewards and avatar features.”

2. Show onboarding

Point out profile/questionnaire and avatar setup. Explain these represent user setup data; do not expose real user data.

3. Show daily quests

Open Quest. Explain target/current progress, quest type, and completion control. Complete a safe sample quest if the demo profile allows it.

4. Show rewards

Show XP, gold, level, achievements, or streak. Explain completion updates local progression state.

5. Show Wardrobe and Shop

Demonstrate owned/equipped items and purchase/reward flow only with demo data; mention online dependencies if relevant.

6. Explain optional tracking

Describe step-sensor or avoidance permissions. Demo live monitoring only if permissions are prepared and behavior is reliable on the test device.

7. Close on scope

Summarize what the app implements, then transition to study findings from the manuscript.

Before defense day: charge the demo device, turn off disruptive notifications, prepare a demo account, verify permissions and network-dependent features, keep screenshots/video as fallback, and rehearse a version that does not depend on live sensor/permission prompts.

9. Team fact sheet: verify from the manuscript

Use the approved final manuscript, particularly for research details and any revisions to the attached print copy.

Manuscript item	Team verified answer
Project title	[Exact approved capstone title]
Research problem	[Statement of the problem]
General objective	[Exact approved objective]
Specific objectives	[List and map to implemented/tested functions]
Target users / locale	[Who, where, inclusion criteria]
Research design	[Design and rationale]
Sampling / sample size	[Method, n, respondent profile]
Evaluation instrument	[Name, sections, validation/reliability]
Testing devices	[Models, OS versions, conditions]
Evaluation method/statistics	[Method and why appropriate]
Key findings	[Exact values, tables, interpretation]
Conclusion	[Conclusion supported by findings]
Limitations	[Research and technical limitations]
Ethics/data privacy	[Consent, data collected, storage, retention, approvals]
Future work	[Prioritized recommendation]

10. Rapid-fire practice and final checklist

Q. What class contains daily quest/business logic?

Suggested response: TaskManager.

Q. What screen hosts the four main product areas?

Suggested response: DashboardActivity: Quest, Wardrobe, Shop, and Me.

Q. What is the primary structured local database?

Suggested response: SQLite through DatabaseHelper/SQLiteOpenHelper.

Q. Which service tracks steps?

Suggested response: StepTrackingService.

Q. Which service monitors selected app usage?

Suggested response: AppMonitorService, with AvoidanceOverlayManager for interruption UI.

Q. Does source code prove the app improves habits?

Suggested response: No. That requires empirical evaluation.

Q. Is all data in Firebase?

Suggested response: No. Primary app state is local; Firebase handles auth and selected cloud features.

Q. What is a known data-upgrade concern?

Suggested response: DatabaseHelper.onUpgrade drops/recreates daily_tasks and task_templates.

Final rehearsal checklist

- Every presenter can explain the purpose in 20 seconds.
- Everyone knows Activity purposes and which dashboard areas are not separate Activities.
- The team can trace quest generation/input through storage, reward calculation, and UI.
- Permission-dependent features and limitations are described accurately.
- Research design, participant, instrument, statistical, and result statements match the manuscript.
- Demo data is safe; account, permissions, network, and backup are ready.
- Presenters separate evidence from future plans.

Source basis and scope

Prepared from the DaGOAL Android project source, including AndroidManifest.xml, app/build.gradle.kts, Activity/service classes, TaskManager.java, DatabaseHelper.java, DatabaseContract.java, and relevant layouts/resources. This is a software walkthrough and defense-preparation aid, not a substitute for the approved capstone manuscript, ethics documentation, test logs, or study findings.

You are ready to explain what you built, show how users experience it, and defend the claims your evidence supports.